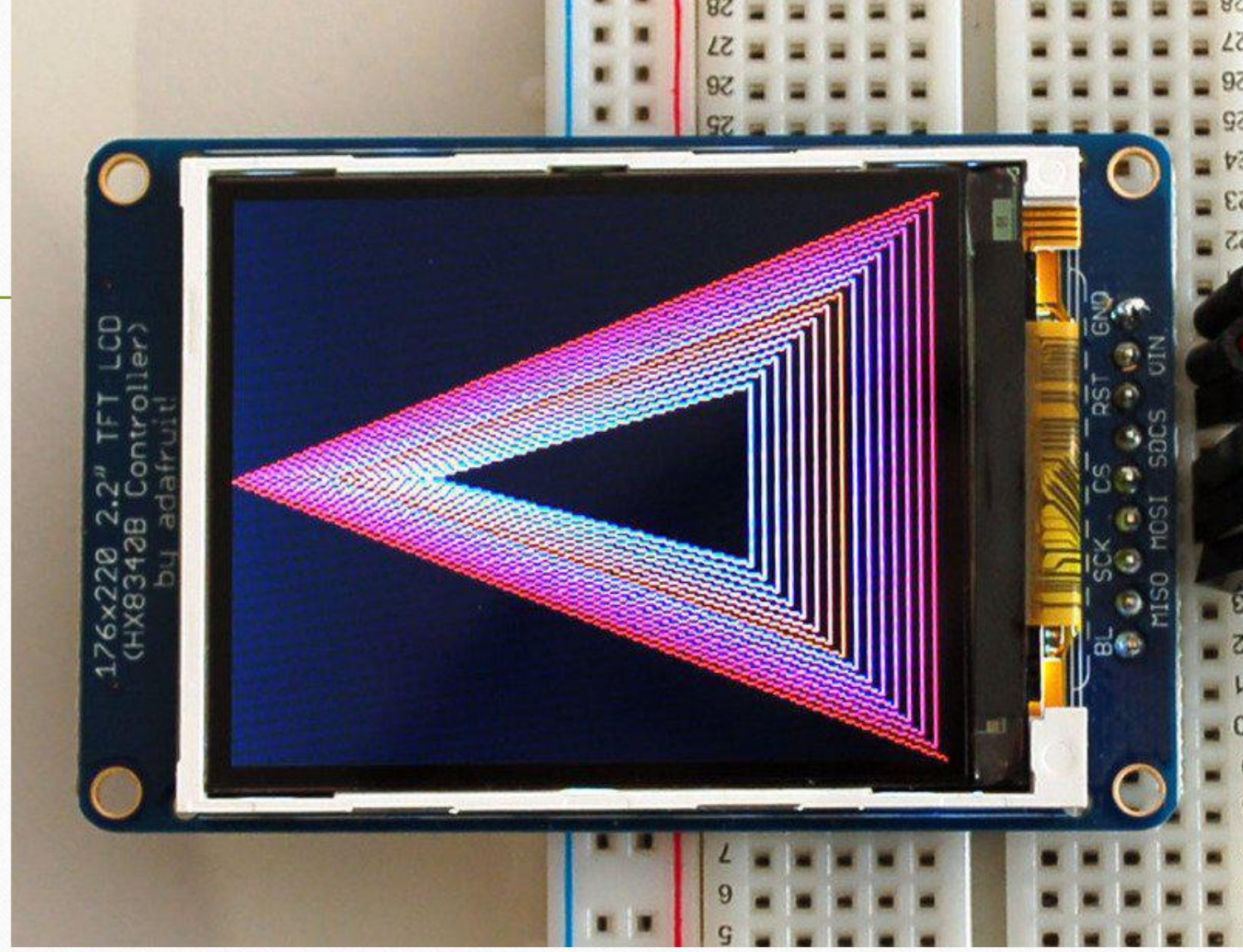


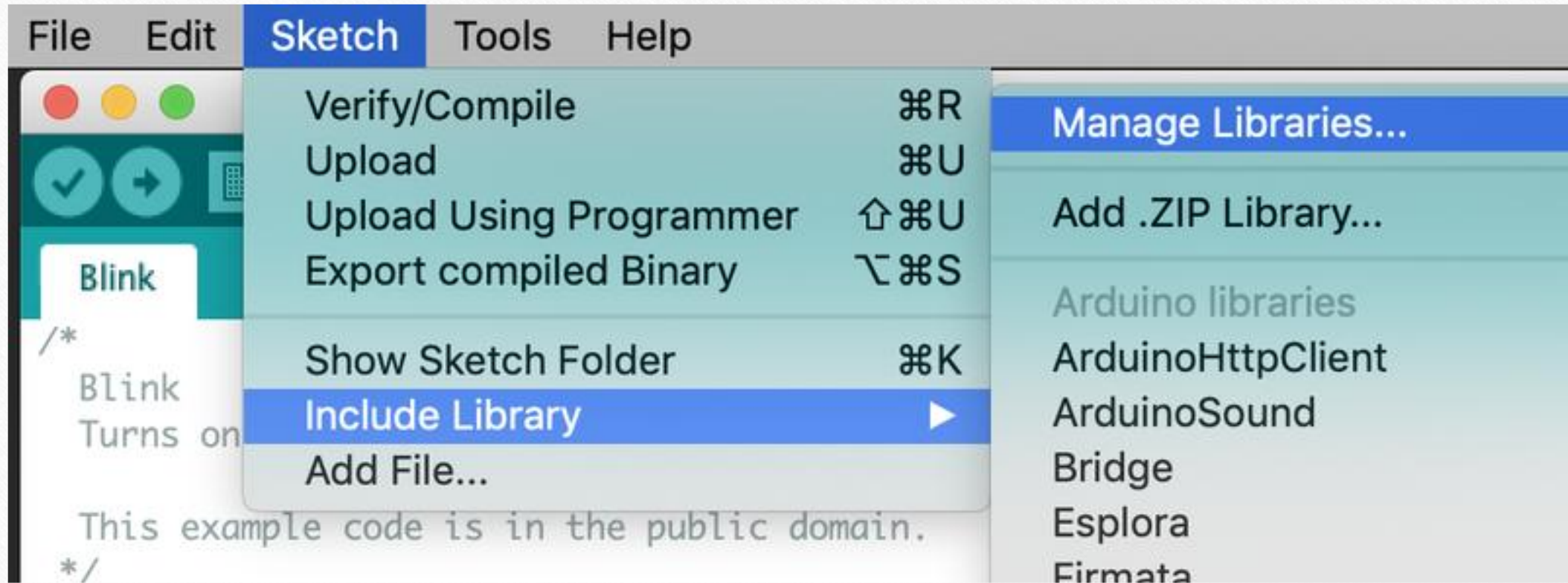
Adafruit GFX Graphics Library

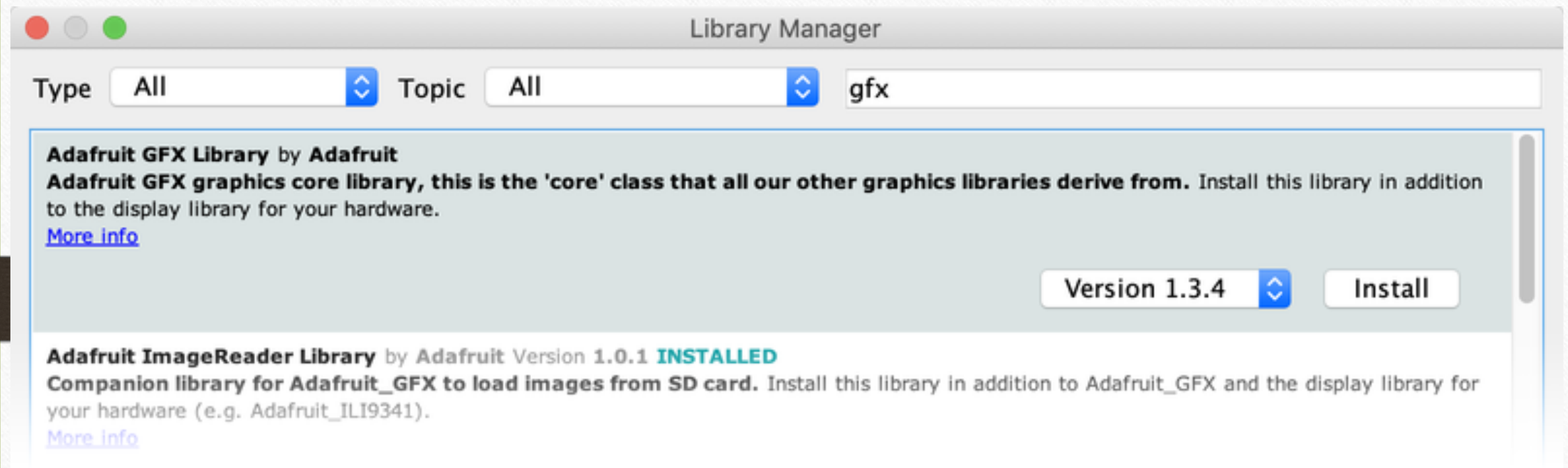
Chao-Chin Wu

<https://learn.adafruit.com/adafruit-gfx-graphics-library>



installed using the Arduino Library Manager





Download Adafruit_GFX Library

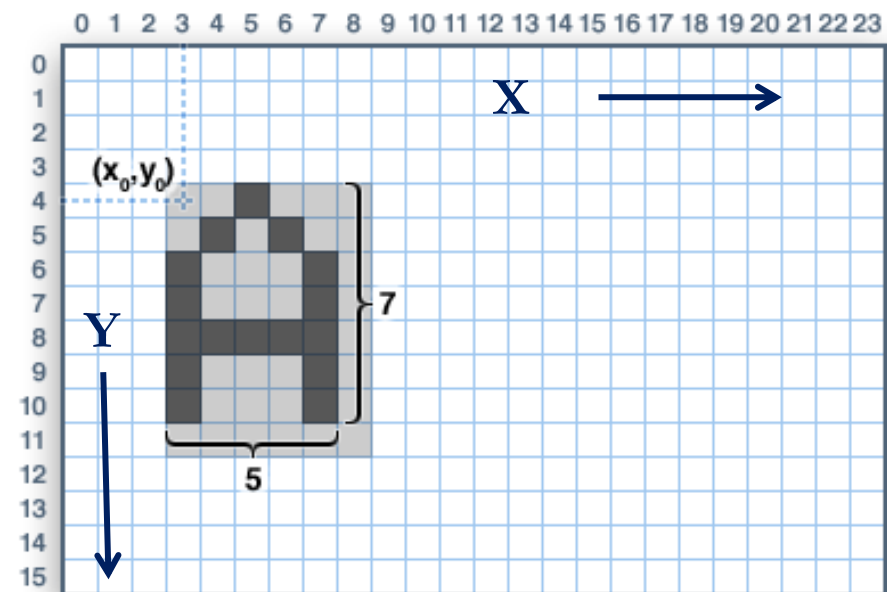
<https://adafru.it/cBB>

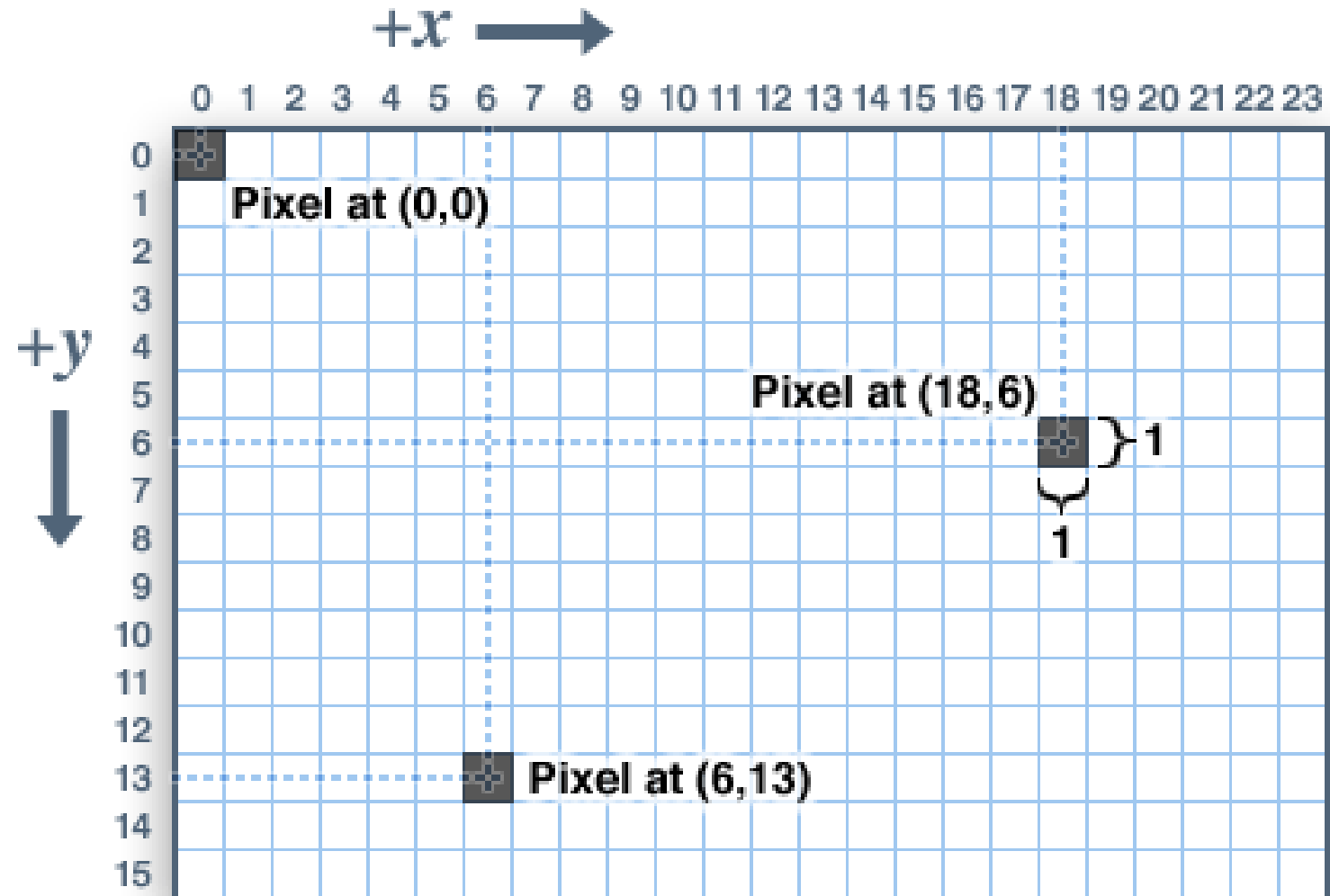
Download Adafruit_BusIO Library

<https://adafru.it/Ldl>

Coordinate System and Units

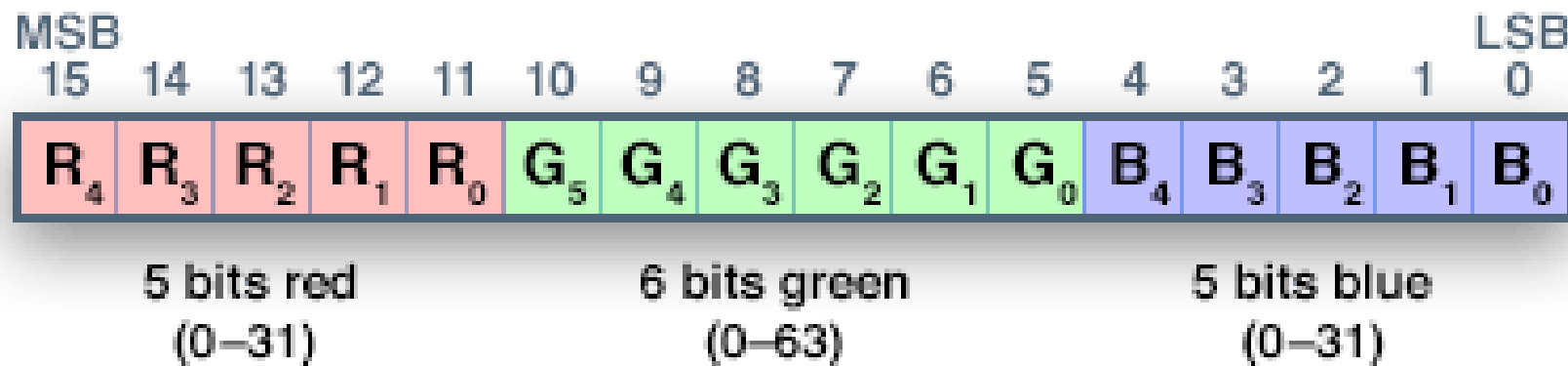
- **Pixels** — picture elements, the blocks comprising a digital image — are addressed by their horizontal (X) and vertical (Y) coordinates.
- The **coordinate system**
 - Default
 - the origin (0,0) at the top left corner
 - One of four rotation settings can also be applied, indicating which corner of the display represents the top left.





-
- The library will safely “clip” any graphics drawn off the edges of the screen.
 - done on purpose sometimes, as with scrolling text displays.
 - **Colors** are represented as unsigned 16-bit values.
 - Some displays may physically be capable of more or fewer bits than this, but the library operates with 16-bit values
 - these are easy for the Arduino to work with while also providing a consistent data type across all the different displays.

- The primary color components — red, green and blue — are all “packed” into a single 16-bit variable
 - That extra bit is assigned to green because our eyes are most sensitive to green light



// Color definitions

#define BLACK 0x0000

#define BLUE 0x001F

#define RED 0xF800

#define GREEN 0x07E0

#define CYAN 0x07FF

#define MAGENTA 0xF81F

#define YELLOW 0xFFE0

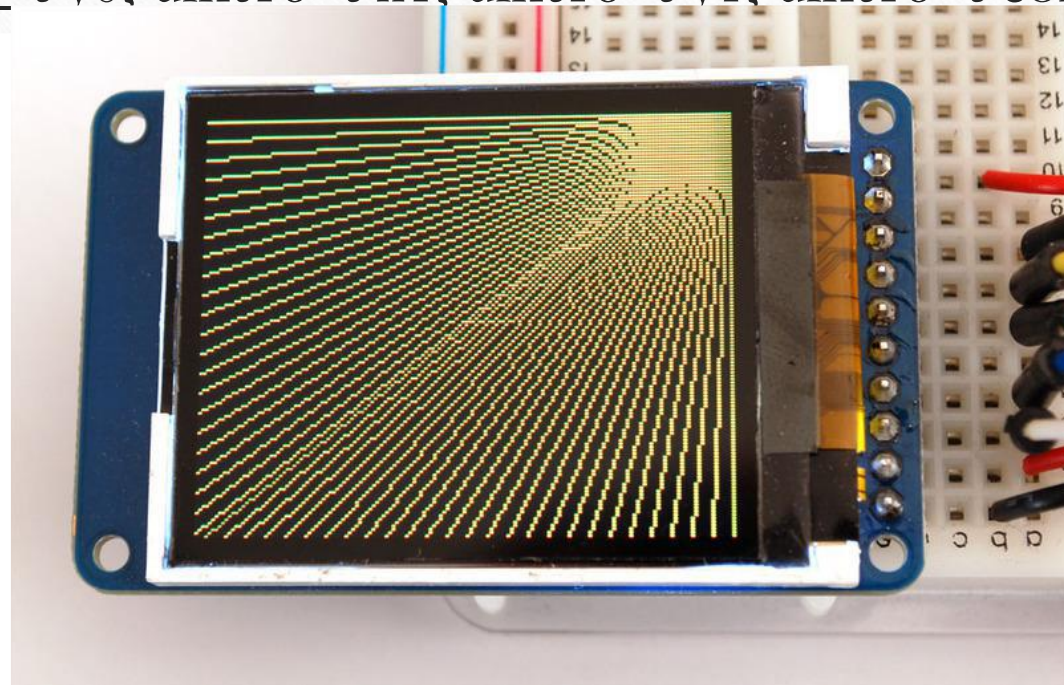
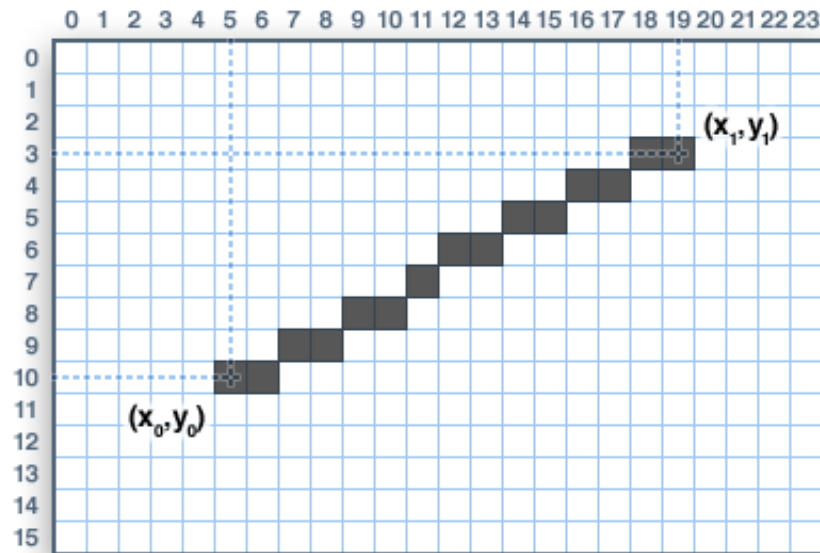
#define WHITE 0xFFFF

Graphics Primitives

- The function descriptions below are merely **prototypes**
 - show `print(1234.56)`
 - your actual code would place the **object name** before this

Drawing lines

```
void drawLine(uint16_t x0, uint16_t y0, uint16_t x1, uint16_t y1, uint16_t color);
```



-
- For **horizontal or vertical lines**, there are optimized line-drawing functions that avoid the angular calculations:

```
void drawFastVLine(uint16_t x0, uint16_t y0, uint16_t length, uint16_t color);
```

```
void drawFastHLine(uint8_t x0, uint8_t y0, uint8_t length, uint16_t color);
```

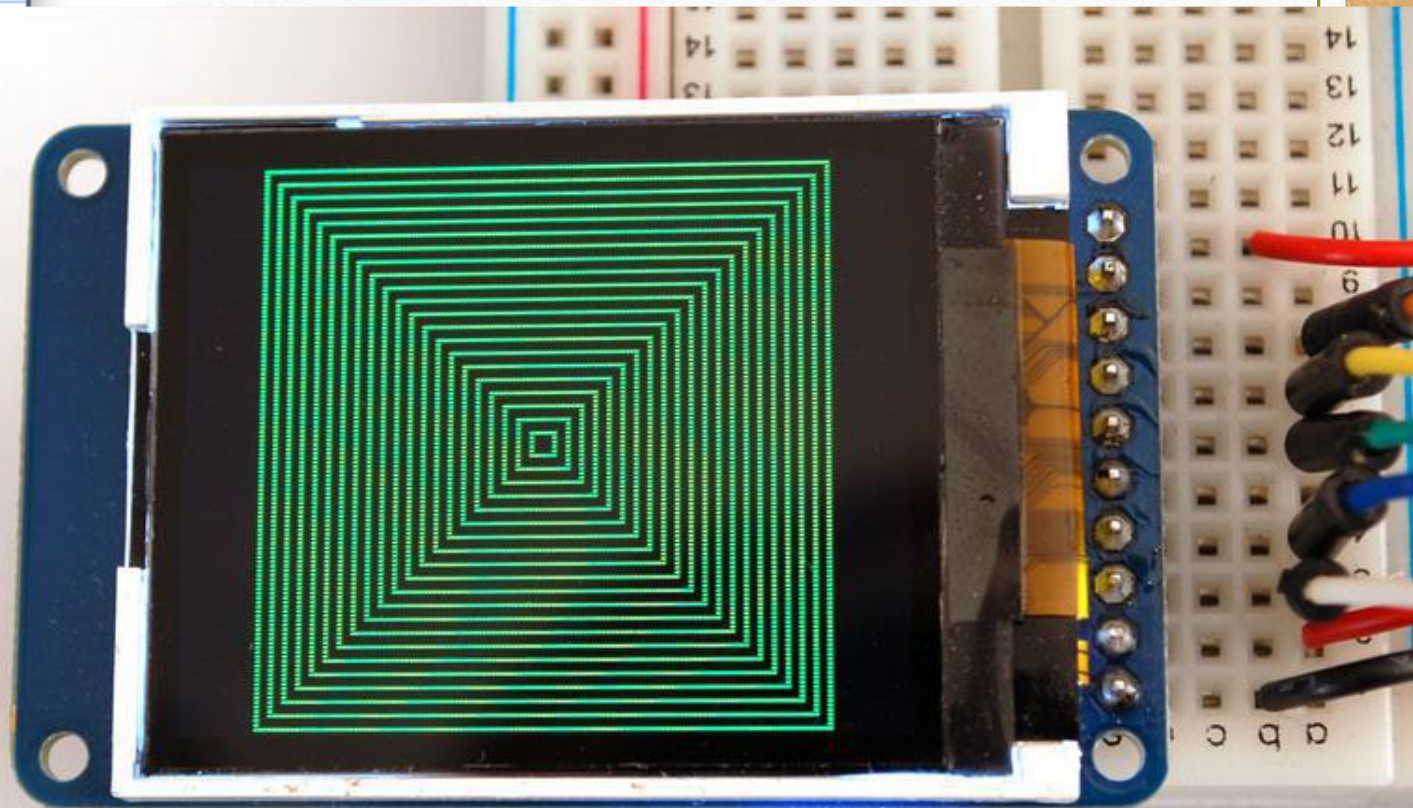
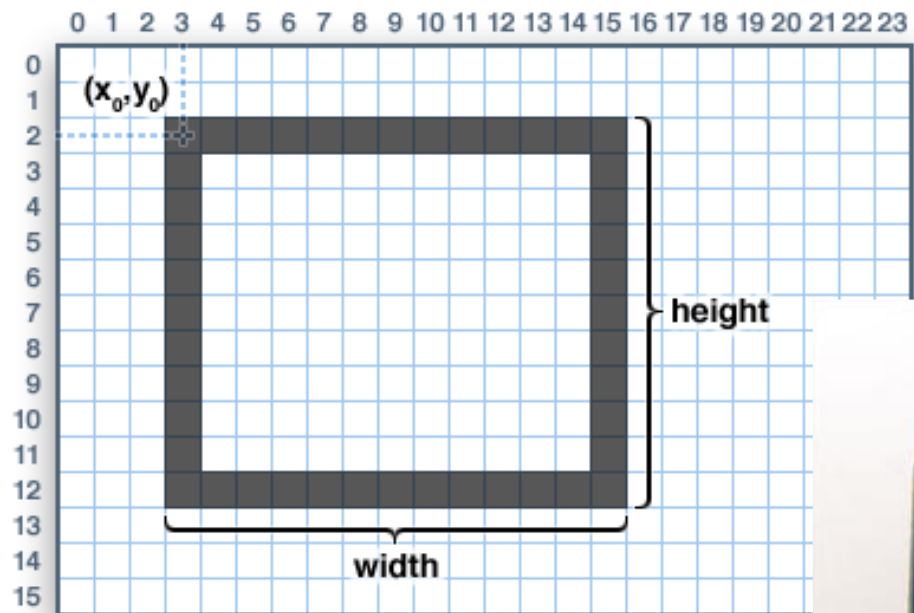
Rectangles

- `drawRect()` renders just the frame (outline) of the rectangle
 - the interior is unaffected

```
void drawRect(uint16_t x0, uint16_t y0, uint16_t w, uint16_t h, uint16_t color);
```

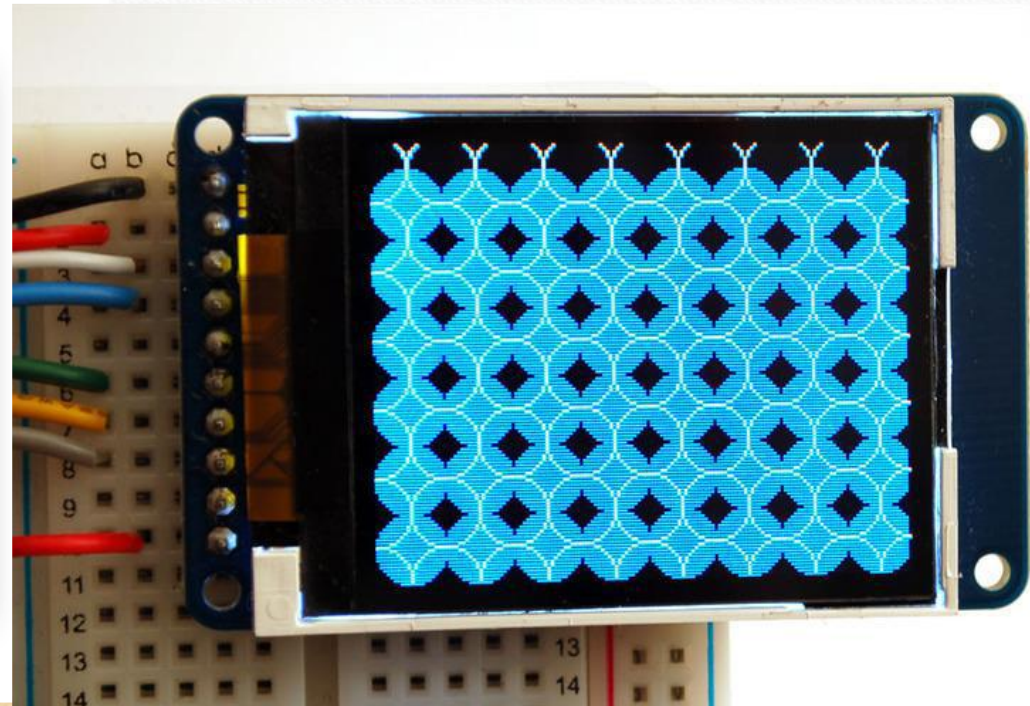
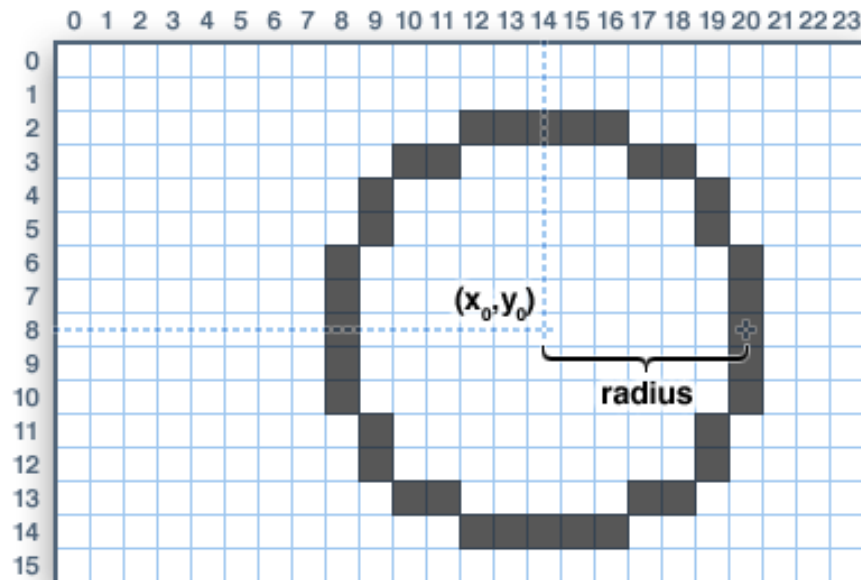
- `fillRect()` fills the entire area with a given color:

```
void fillRect(uint16_t x0, uint16_t y0, uint16_t w, uint16_t h, uint16_t color);
```



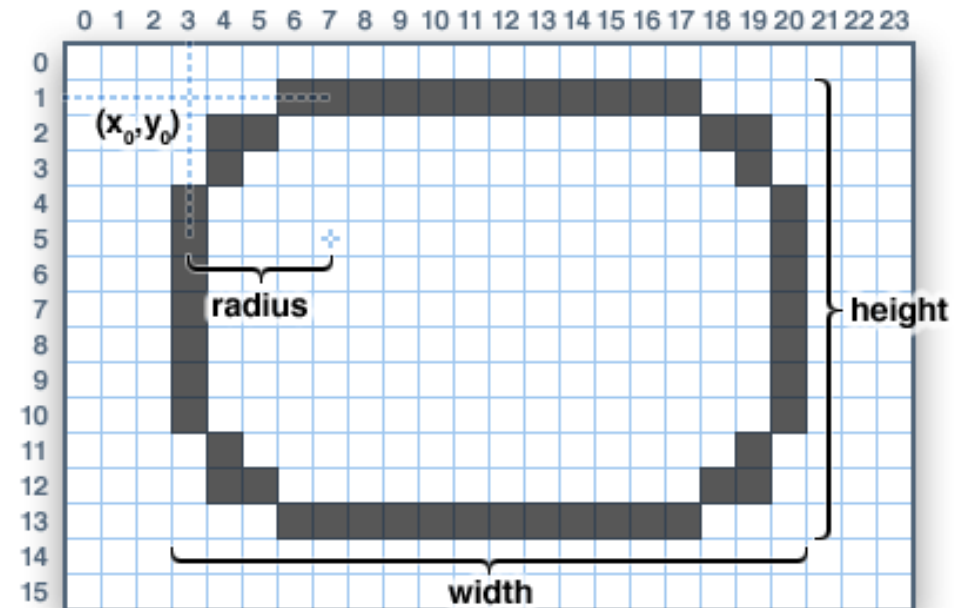
Circles

- `void drawCircle(uint16_t x0, uint16_t y0, uint16_t r, uint16_t color);`
- `void fillCircle(uint16_t x0, uint16_t y0, uint16_t r, uint16_t color);`



Rounded rectangles

- void **drawRoundRect**(uint16_t x0, uint16_t y0, uint16_t w, uint16_t h, uint16_t radius, uint16_t color);
- void **fillRoundRect**(uint16_t x0, uint16_t y0, uint16_t w, uint16_t h, uint16_t radius, uint16_t color);

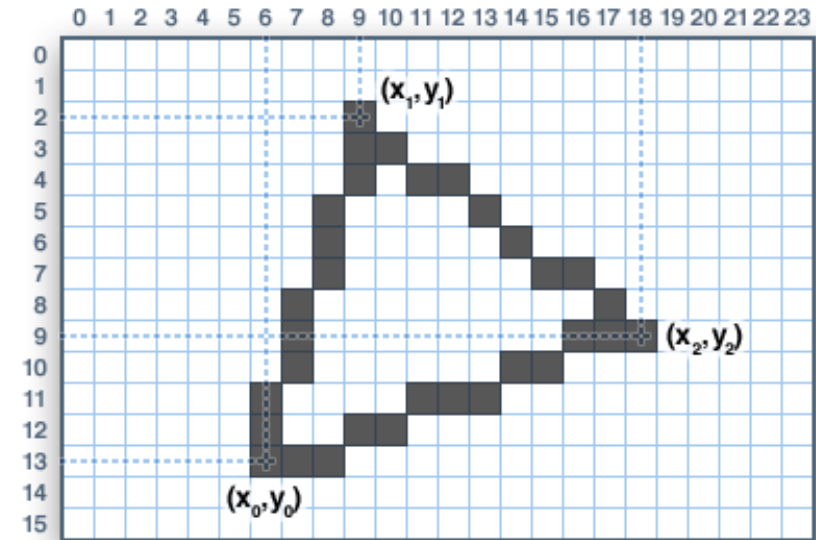


Note

- because the **circle** functions are always drawn relative to a **center pixel**, the resulting circle diameter will always be an **odd number** of pixels.
- If an **even-sized circle** is required (which would place the center point between pixels), this can be achieved using one of **the rounded rectangle** functions:
 - pass an **identical width and height** that are even values,
 - and a **corner radius** that's **exactly half this value**.

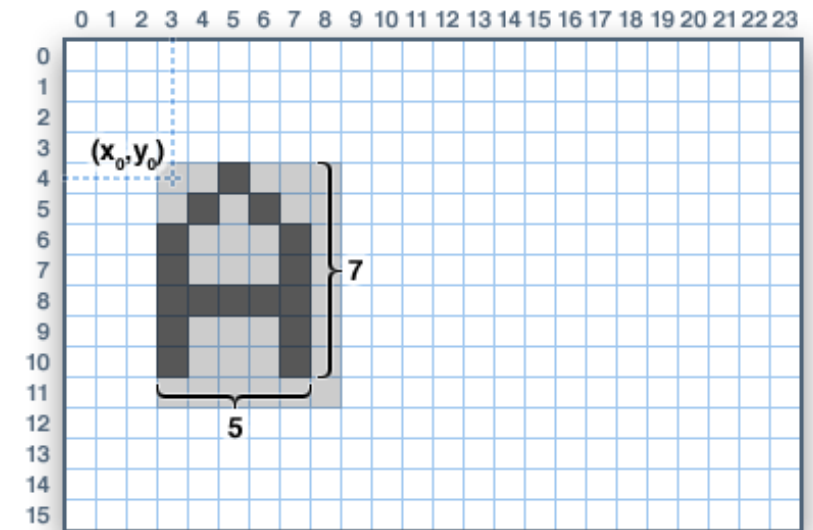
Triangles

- `void drawTriangle(uint16_t x0, uint16_t y0, uint16_t x1, uint16_t y1, uint16_t x2, uint16_t y2, uint16_t color);`
- `void fillTriangle(uint16_t x0, uint16_t y0, uint16_t x1, uint16_t y1, uint16_t x2, uint16_t y2, uint16_t color);`



Characters and Text

- Two basic string drawing procedures
 - just for a single character



```
void drawChar(uint16_t x, uint16_t y, char c, uint16_t color, uint16_t bg, uint8_t size);
```

- Text

- Instead of one procedure, the text size, color and position are set up in separate functions, and then
- the `print()` function is used

```
void setCursor(int16_t x0, int16_t y0);  
void setTextColor(uint16_t color);  
void setTextColor(uint16_t color, uint16_t backgroundcolor);  
void setTextSize(uint8_t size);  
void setTextWrap(boolean w);
```

-
- `setCursor(x, y)`
 - place the top left corner of the text
 - Initially this is set to (0,0)
 - `setTextColor(color)`
 - set the text color with
 - by default this is **white**
 - a background color can be specified as an **optional** second parameter to `setTextColor()`.

-
- `setTextSize(size)` will multiply the scale of the text by a given integer factor.
 - After setting everything up, use `print()` or `println()`
 - just like you do with Serial printing (<https://adafru.it/UdO>)!

-
- By default, long lines of text are set to automatically “wrap” back to the leftmost column.
 - To override this behavior , use `setTextWrap(false)`.
 - text will run off the right side of the display
 - The normal wrapping behavior is restored with `setTextWrap(true)`.

Bitmaps

- `void drawBitmap(int16_t x, int16_t y, uint8_t *bitmap, int16_t w, int16_t h, uint16_t color);`
 - a contiguous block of bits to the display,
 - where each '1' bit sets the corresponding pixel to 'color,'
 - while each '0' bit is skipped.
 - x, y is the top-left corner where the bitmap is drawn,
 - w, h are the width and height in pixels.

-
- The bitmap data must be located in program memory using the **PROGMEM** directive.
 - This is a somewhat advanced function and beginners are best advised to come back to this later.
 - For an introduction, see the Arduino tutorial on PROGMEM usage (<https://adafru.it/aMw>).
 - Here's a handy webtool for generating bitmap -> memorymaps (<https://adafru.it/l3b>)

Clearing or filling the screen

- The `fillScreen()` function will set the entire display to a given color, erasing any existing content:

```
void fillScreen(uint16_t color);
```

Rotating the Display

- change the coordinate system for any new drawing
 - this will not rotate what you already drew
- We can only rotate 0, 90, 180 or 270 degrees

```
void setRotation(uint8_t rotation);
```

-
- The rotation parameter can be 0, 1, 2 or 3.
 - For displays that are part of an Arduino shield,
 - rotation value 0 sets the display to a portrait (tall) mode, with the USB jack at the top right.
 - Rotation value 2 is also a portrait mode, with the USB jack at the bottom left.
 - Rotation 1 is landscape (wide) mode, with the USB jack at the bottom right, while
 - rotation 3 is also landscape, but with the USB jack at the top left.

-
- When rotating, the origin point $(0,0)$ changes
 - it should be arranged at the top-left of the display for the other graphics functions to make consistent sense

-
- If you need to reference the size of the screen (which will change between portrait and landscape modes), use `width()` and `height()`.

```
uint16_t width();  
uint16_t height();
```